Build a Financial Controller Agent with Codex



Jayeeta Putatunda

FDE AI engineering Lead, **Turing; Women in AI NY** State Ambassador

NVIDIA NEPA Advisor | **HearstLab** VC Scout | **AI100 awardee**

Mentor @ Money20/20, **Top 1% NA** Data Coach at Topmate

Previous session: Agents for Financial Analysis



Youtube Link: https://www.youtube.com/live/77aMTfqdlu4

Codex Inside the Workflow

Codex operates across desktop, CLI, IDE, and cloud environments. It reads your repository, invokes its tools, runs the checks, and revises its own work — all within a structured harness you define. This session walks through how that harness is built, who owns each decision, and what prevents bad outcomes from reaching production.



Codex

Repo Context

Reads files, dependencies, and history

Tool Use

Search, retrieval, calculators, and gates

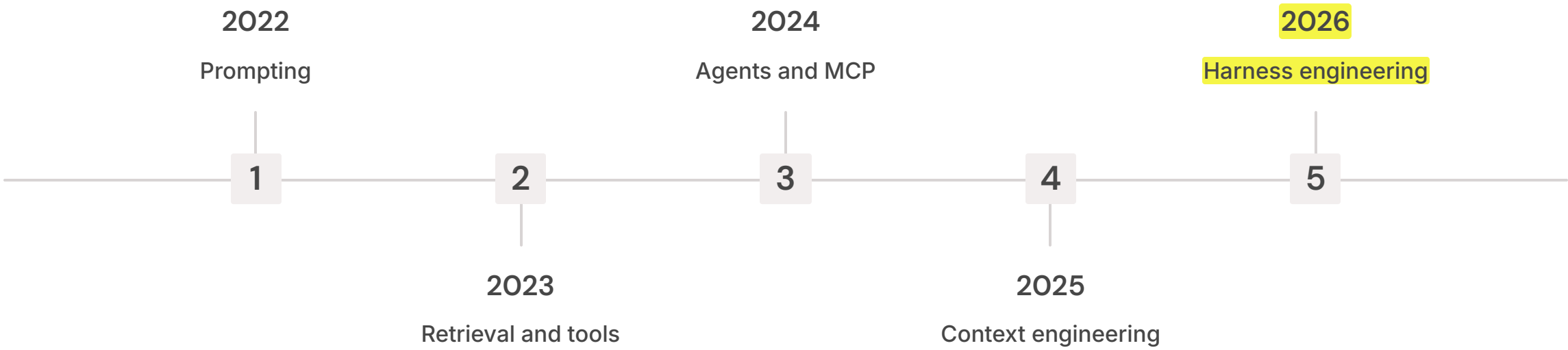
Test Loop

Runs checks and revises the work

> "I'm building the harness. Codex writes the implementation."

The Path to Harness Engineering

These labels mark shifts in industry attention. Each step gave models access to more context and more actions. Once agents could work for longer stretches, the surrounding system became the binding constraint.



~1 Million Lines

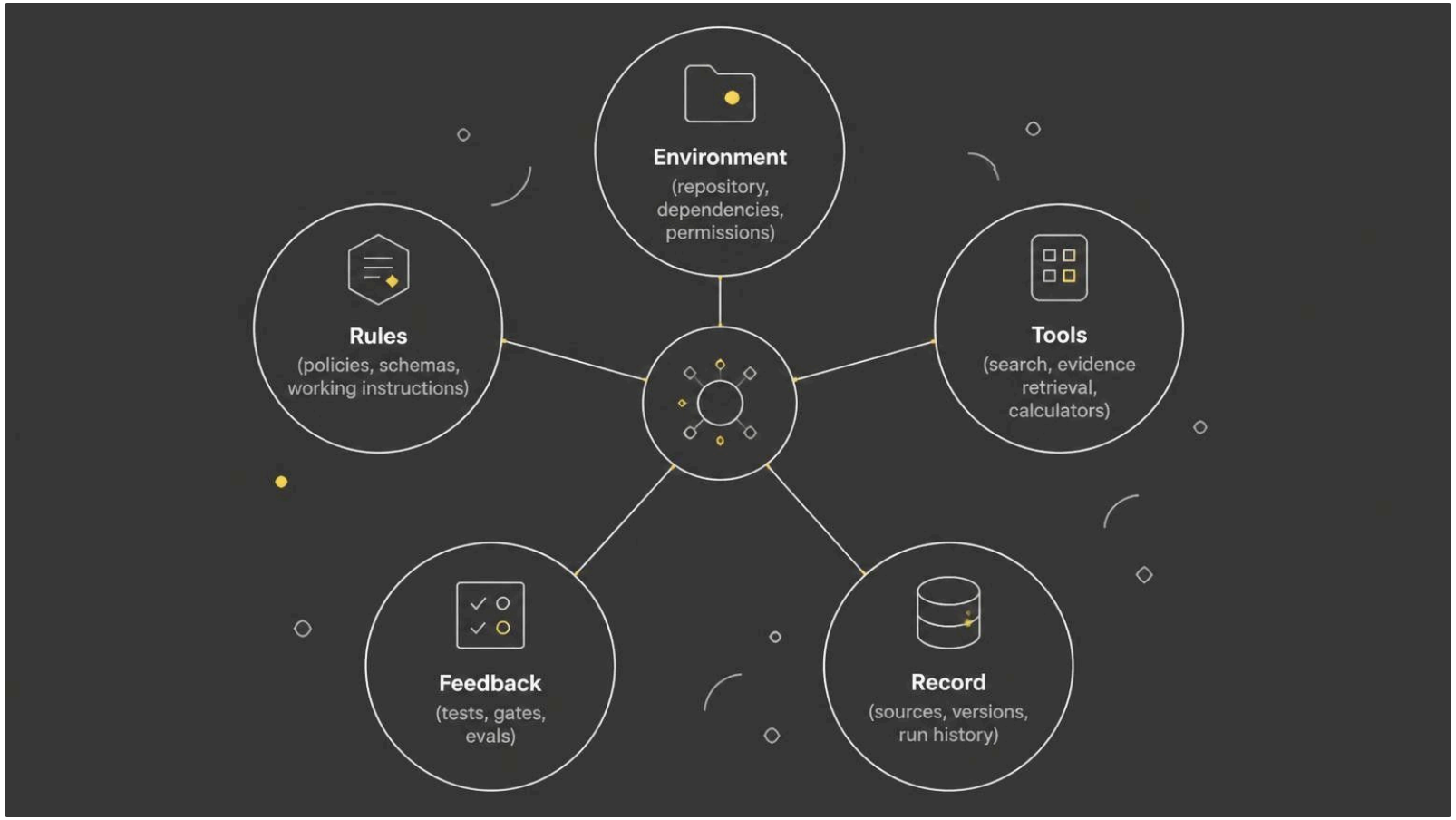
Lines of Codex-written code produced in OpenAI's internal experiment - the largest public signal of agentic output at scale.

Why Progress Stalled Early

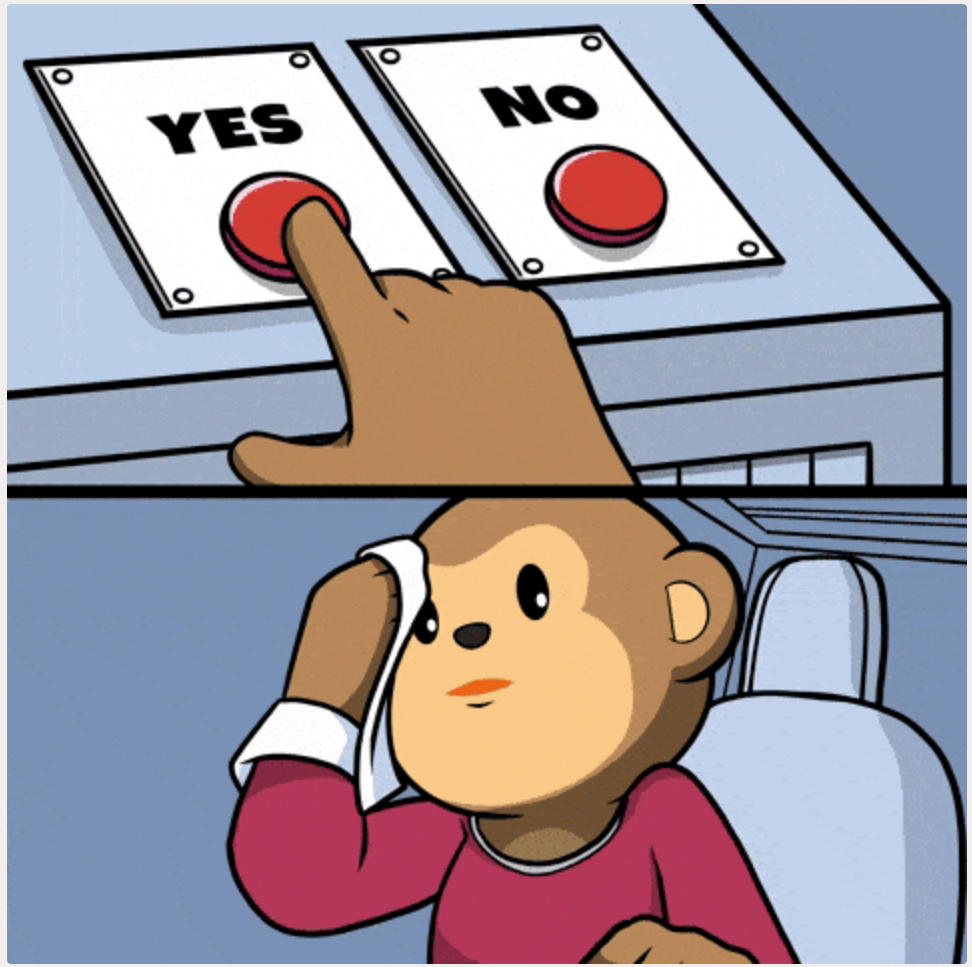
OpenAI's team reported that early gains slowed not because the model lacked capability, but because the **environment lacked structure** - missing tools, missing feedback loops, missing working instructions. The harness was the missing layer. OpenAI harness engineering →

What a Harness Contains

The model sits *inside* the harness. The harness decides what the model can see, which tools it may invoke, how its work gets checked, and what evidence survives after the run. MCP belongs inside the tools layer. Context engineering spans the environment, rules, and evidence. Harness engineering connects the whole system.



Deterministic where we can, agentic where we must - evals decide what moves to next step.



The New Moat: Decision-Making When AI Does the Rest

The Finance Operating Model

Codex gets the work that benefits from flexible investigation. Deterministic code keeps the jobs where repeatability and auditability matter most. The controller retains the accounting decision - always. This division is not a convenience; it is the ownership model baked into the implementation.

Deterministic Code

- Admits the data
- Calculates the amounts
- Applies control thresholds

Codex

- Investigates the exception
- Finds current evidence
- Explains the result

Controller

- Judges the accounting treatment
- Approves or rejects the draft
- Owns posting

> **Historical guidance directs the search. Current records prove the claim.**

Two Exceptions to Handle:

SYNTHETIC DATA

Two exceptions, two distinct paths. The first case has an evidence gap, so Codex investigates. The second already carries enough information for a coded control to escalate without model involvement. The system does not ask a model to invent a diagnosis where one isn't needed.

Annual Software Bill — Investigation Path

Inputs: $144k September expense · $12k monthly budget · 12-month current contract

Codex retrieves the proof: Invoice · Contract · Prepaid policy

Code calculates: $12k September expense · $132k prepaid draft

Route: REVIEW — Human decision: **PENDING**

Evidence gap present → Codex investigates, code calculates, controller decides.

Aging Receipts — Coded Control Path

Inputs: 3 open checks · $12.4k unmatched · Oldest receipt: 17 days · Close: 3 business days away

Code applies the threshold

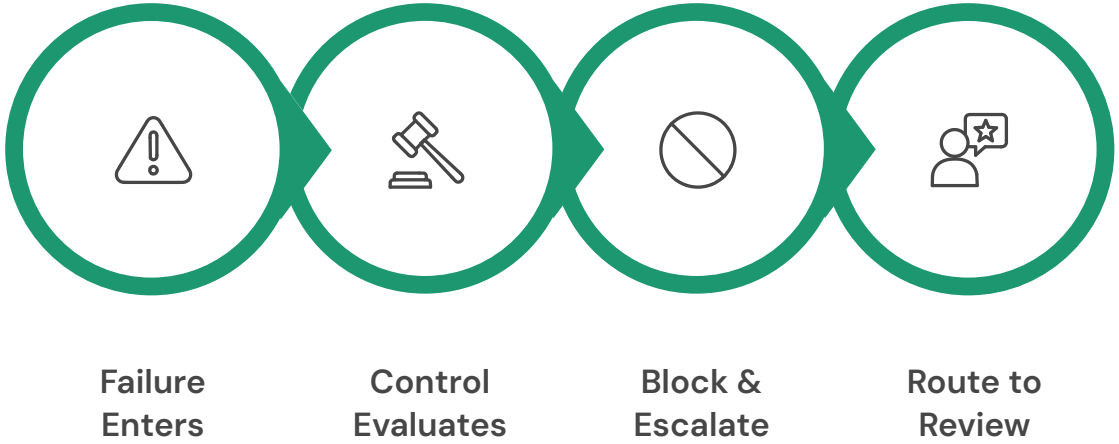
Route: ESCALATE — Cause remains unresolved

Sufficient information already present → no agent required. Coded control escalates directly.

The contrast between these two paths is the point: not every exception needs an agent. The harness routes each case to the right mechanism.

Failure Boundaries and Release Gate

The run record tells us how much work happened. The gates tell us whether the result can reach a controller. The evals decide whether the next version ships. Five replay evals gate every release — no eval pass, no deploy.



Failure Boundaries

Tampered source package Blocked before Codex is invoked	**Missing current contract** Partial result returned with escalation	**Unsupported figure** Claim audit fails- result rejected	**Draft accounting action** Kept unposted and pending controller review

Live-Run Evidence

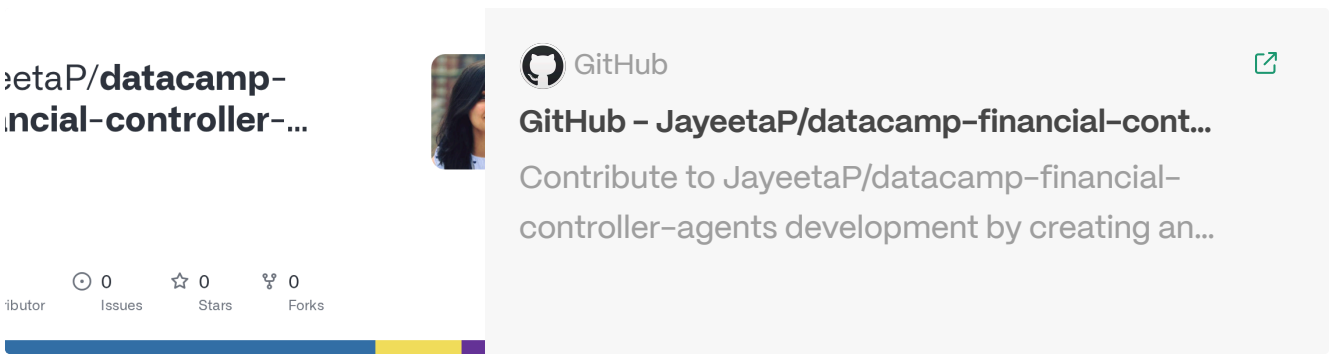
29s Run time	**155k** Input tokens	**132k** Cached tokens
1.3k Output tokens	**8** Governance gates	

85% cache hit rate on a 155k-token context window. This is the token-economics story — drawn from a working demonstration, not a general claim about falling model prices.



Let's Get Coding!

GitHub Repo Link: https://github.com/JayeetaP/datacamp-financial-controller-agents/tree/main



Conclusion: Problem-Driven AI

Focus on Problems

Start with specific business needs.

Evaluate and Learn

Design agents for shifting goals.

Simplify Complexity

Leverage frameworks to streamline.



Q&A